

Flutter and Dart Fundamentals

IS2205 - Mobile Application Design &
Development

What is Flutter?

- Open-source UI toolkit by Google
- Single codebase → Android, iOS, Web, Desktop
- Fast development (Hot reload)
- Rich UI with customizable widgets

Flutter and cross-platform development :

— Highlights

- One Codebase - For mobile(Android/iOS),web and desktop using the Dart programming language
- Platform Integration - Provides APIs to interact with platform-specific features and access device features like the camera, sensors etc..
- Consistent User Experience - UI is consistent across platforms, users will have a similar experience regardless of the device used

Flutter and cross-platform development (+)

-
- **Faster Development** : A single codebase significantly speeds up the dev. process. Changes can be applied across platforms simultaneously, reducing duplication of effort.
 - **Cost-Efficiency** : Multi-platforms dev. usually requires separate teams and codebases, increasing costs.
 - **Easier Maintenance** : A single codebase means fewer bugs to fix and updates to roll out. This makes maintenance more streamlined and efficient.

Flutter and cross-platform development (+)

- Rapid Prototyping : Bolstered by Hot Reload allows for rapid prototyping and experimentation.
- Access to New Platforms : As Flutter supports new platforms (embedded devices), existing codebase can be used to target these platforms without a complete rewrite.
- Community and Resources : Flutter has a growing community and an abundance of resources, including documentation, tutorials, and plugins, making it easier to learn and develop with the framework.

Flutter and cross-platform development (-)

- ☐ Large App Size: Compared to native apps (with the inclusion of Flutter engine and framework libraries) can suffer on limited storage space.
- Performance Trade-offs: While Flutter compiles to native machine code, highly intensive hardware tasks (like heavy 3D rendering) are still better suited for raw native code (Kotlin/Swift) .

Flutter and cross-platform development (-)

- ☐ Limited Native Features : Flutter provides plugins to access native device features, yet some certain platform-specific functionality may not be supported.
- Learning Curve : Newcomers to Dart programming or the widget-based UI paradigm a learning curve may come. Developers coming from other programming languages or frameworks may need time to adjust to Flutter's architecture and concepts.

Flutter vs. Native Development

- Feature	Flutter	Native (Android/iOS)
Codebase	Single (Dart)	Separate (Kotlin/Swift)
Performance	Equal or Less than native	Native
UI Consistency	Same across platforms	Different look & feel

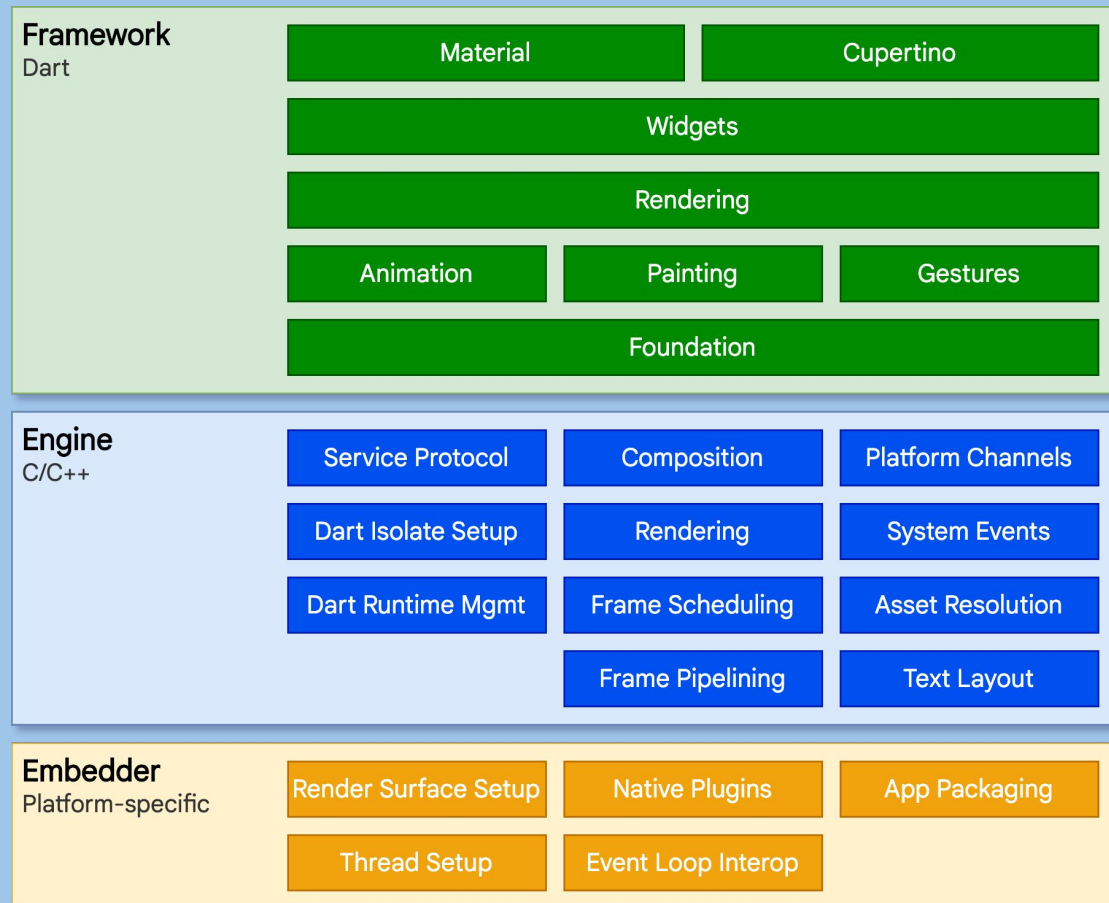
Flutter Architecture

An extensible, layered system

A set of independent libraries that each depend on the underlying layer

No layer has privileged access to the layer below

Parts in the framework level are designed to be optional and replaceable



Flutter Architecture : Summary

- Framework – Widgets, rendering, Dart libraries
- Engine – Impeller graphics, text, C++ core
- Embedder – Platform-specific integrations (Android/iOS)

Comparing to Android Platform?

Introduction to Dart

- Modern, object-oriented language
- Compiles to native ARM & JavaScript
- Null safety and async/await support
- Strongly typed but flexible (type inference and *dynamic*)

Dart Syntax Basics

- Types: `var`, `int`, `String`, `double`, `bool`
- Constants: `final` (run-time), `const` (compile-time) set once
- Operators: `+`, `-`, `*`, `/`, `~/` (integer div.), `%` (modulo)
- Comments : `//` `/**`

//Example:

```
var name = "Kamal";  
int age = 29;  
double gpa = 3.5;  
bool passed = true;
```

Control Flow

// if / else

//Example conditional statement:

```
int age = 20;
if (age >= 18) {
    print('You are an adult.');
```

```
} else if (age >= 13) {
    print('You are a teenager.');
```

```
} else {
    print('You are a child.');
```

```
}
```

Control Flow

//for loops

//for loop with index variable:

```
for (int i = 0; i < 5; i++) {  
    print(i);  
}
```

Control Flow

//for loops

// for loop with 'in' :

```
List<String> fruits = ['Apple',  
    'Banana', 'Orange'];  
    for (String fruit in fruits) {  
        print('Fruit: $fruit');  
    }
```

Control Flow

//while loops

//Example while loop:

```
int count = 1;
while (count <= 3) {
    print('While loop: $count');
    count++;
}
```

Control Flow

//while loops

//Example do-while loop:

```
int count = 1;
do {
    print('Do-while loop: $count');
    count++;
} while (count <= 3);
```

Online runtime for Dart

<https://dartpad.dev/>

Functions in Dart

//Simple function:

```
int add(int a, int b) {  
    return a + b;  
}
```

//Arrow function:

```
int square(int x) => x * x;
```

Basic Exception Handling

//Use try-catch

//Example:

```
try {  
    var result = 10 ~/ 0;  
} catch(e) {  
    print("Error: $e");  
}finally {print("done");}
```

Setting up

- Install Flutter SDK
- Install Android Studio or VS Code
- Setup emulator or connect device

Run first Dart program:

```
void main() {  
    print("Hello, Flutter!");  
}
```

Activity

Write a program in Dart either to:

- check if the given string is a palindrome

- or

- find the nth fibonacci number

Installing Flutter SDK

Visit the official Flutter website (<https://flutter.dev/>) and follow the installation instructions for your operating system (Windows, macOS, Linux).

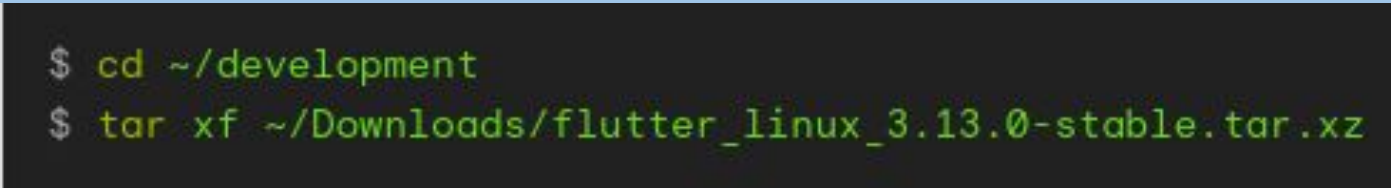
Installing Flutter manually

- Download the installation bundle to get the latest stable release of the Flutter SDK:

A screenshot of a file download link. It shows a blue rectangular button with the text "flutter_linux_3.13.0-stable.tar.xz" in white. The button is centered within a larger white rectangular area.

flutter_linux_3.13.0-stable.tar.xz

- Extract the file in the desired location. For example:

A screenshot of a terminal window with a dark background. It shows two lines of text in a light green monospace font. The first line is a command to change the directory to ~/development, and the second line is a command to extract the tar.xz file from the Downloads directory.

```
$ cd ~/development
$ tar xf ~/Downloads/flutter_linux_3.13.0-stable.tar.xz
```

Installing Flutter manually

—
Add the flutter tool to your path (current terminal):

```
$ export PATH="$PATH: `pwd` /flutter/bin"
```

To permanently add Flutter to your path:

```
~/Desktop/flutter$ vim ~/.bashrc  
export PATH="$PATH:/your-path-to-flutter/flutter/bin"
```

Check the PATH variable and issue “which flutter” command to verify the installation

Installing Dart SDK

Flutter SDK accompanies a compatible version of Dart

If you downloaded the Dart SDK separately two versions might not be compatible!

If the Flutter and Dart commands are from the same bin directory it can be checked as follows. In Windows 'where' command

```
DELLE5440:~/Desktop/flutter$ which flutter dart
~/Desktop/flutter/flutter/bin/flutter
~/Desktop/flutter/flutter/bin/dart
```

Setting up Android Environment

1. Flutter requires Android SDK, Android SDK Command-line Tools, and Android SDK Build-Tools when developing for Android.
2. Run flutter doctor to confirm that Flutter has located your installation of Android Studio.
3. If Flutter cannot locate it, run flutter config --android-studio-dir=<directory> to set the directory that Android Studio is installed to.

```
:~/Desktop/flutter$ flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.13.0, on Ubuntu 22.04 LTS 5.15.0-25-generic,
    locale en_US.UTF-8)
[✓] Android toolchain - develop for Android devices (Android SDK version 33.0.2)
[✓] Android Studio (version 2022.2)
[✓] VS Code (version 1.77.0)
[✓] Connected device (3 available)
[✓] Network resources
```

Setting up Android Environment...

Flutter doctor can point out and guide you to resolve the problems in the installation such as giving the command to execute..

```
bash-5.2$ flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.13.0, on macOS 12.7.5 21H1222 darwin-x64, locale en-LK)
[!] Android toolchain - develop for Android devices (Android SDK version 34.0.0)
    ! Some Android licenses not accepted. To resolve this, run: flutter doctor --android-licenses
[x] Xcode - develop for iOS and macOS
    x Xcode installation is incomplete; a full installation is necessary for iOS and macOS development.
      Download at: https://developer.apple.com/xcode/download/
      Or install Xcode via the App Store.
      Once installed, run:
        sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
        sudo xcodebuild -runFirstLaunch
    x CocoaPods not installed.
      CocoaPods is used to retrieve the iOS and macOS platform side's plugin code that responds to your pl
      Without CocoaPods, plugins will not work on iOS or macOS.
      For more info, see https://flutter.dev/platform-plugins
      To install see https://guides.cocoapods.org/using/getting-started.html#installation for instructions.
[✓] Chrome - develop for the web
[✓] Android Studio (version 2022.2)
```

Setting up IDE

- Android Studio and Visual Studio Code are popular choices for Flutter development
- You can use any text editor to write Flutter code
- As in VS Code and Android Studio plugins/extensions will be useful

Setting up IDE



Flutter

v3.70.0

Dart Code  dartcode.org |  6,690,098 | ★★★★★ (76)

Flutter support and debugger for Visual Studio Code.

Installing 

[DETAILS](#) [FEATURE CONTRIBUTIONS](#) [CHANGELOG](#) [DEPENDENCIES](#)

Introduction

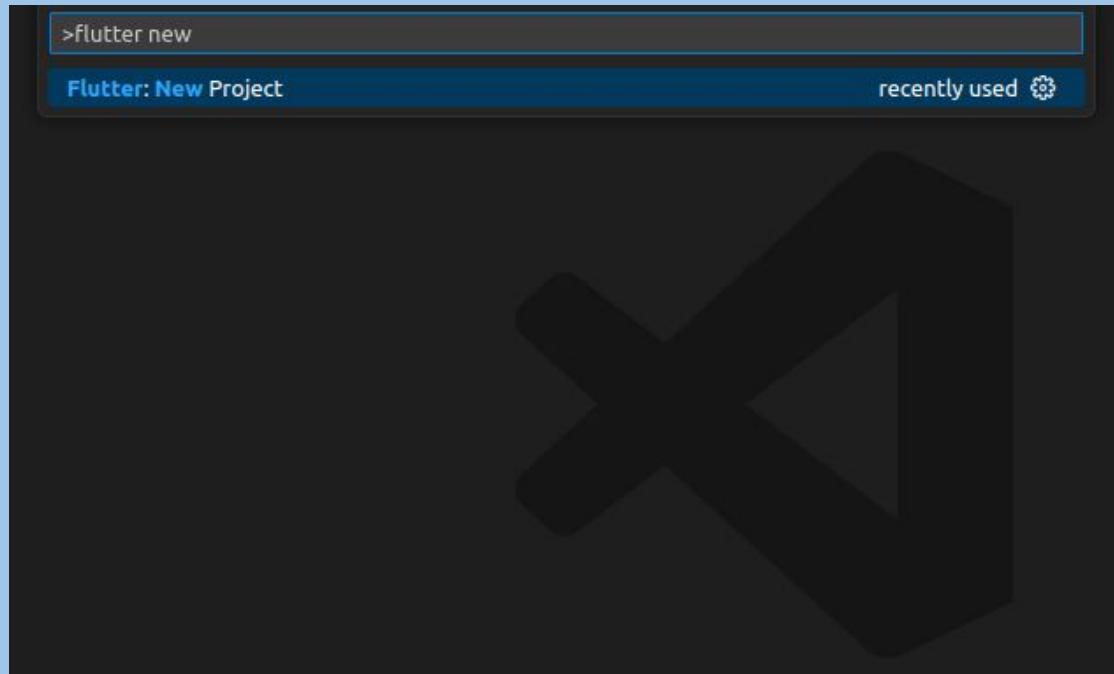
This [VS Code](#) extension adds support for effectively editing, refactoring, running, and reloading [Flutter](#) mobile apps. It depends on (and will automatically install) the [Dart extension](#) for support the [Dart](#) programming language.



Creating Your First Flutter App

- Creating a new Flutter project
- Understanding project structure
- Running the app

Creating from command pallet



Creating from terminal

```
[bash-5.2$ flutter create flutter1
Creating project flutter1...
Resolving dependencies in flutter1... (2.0s)
Got dependencies in flutter1.
Wrote 129 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

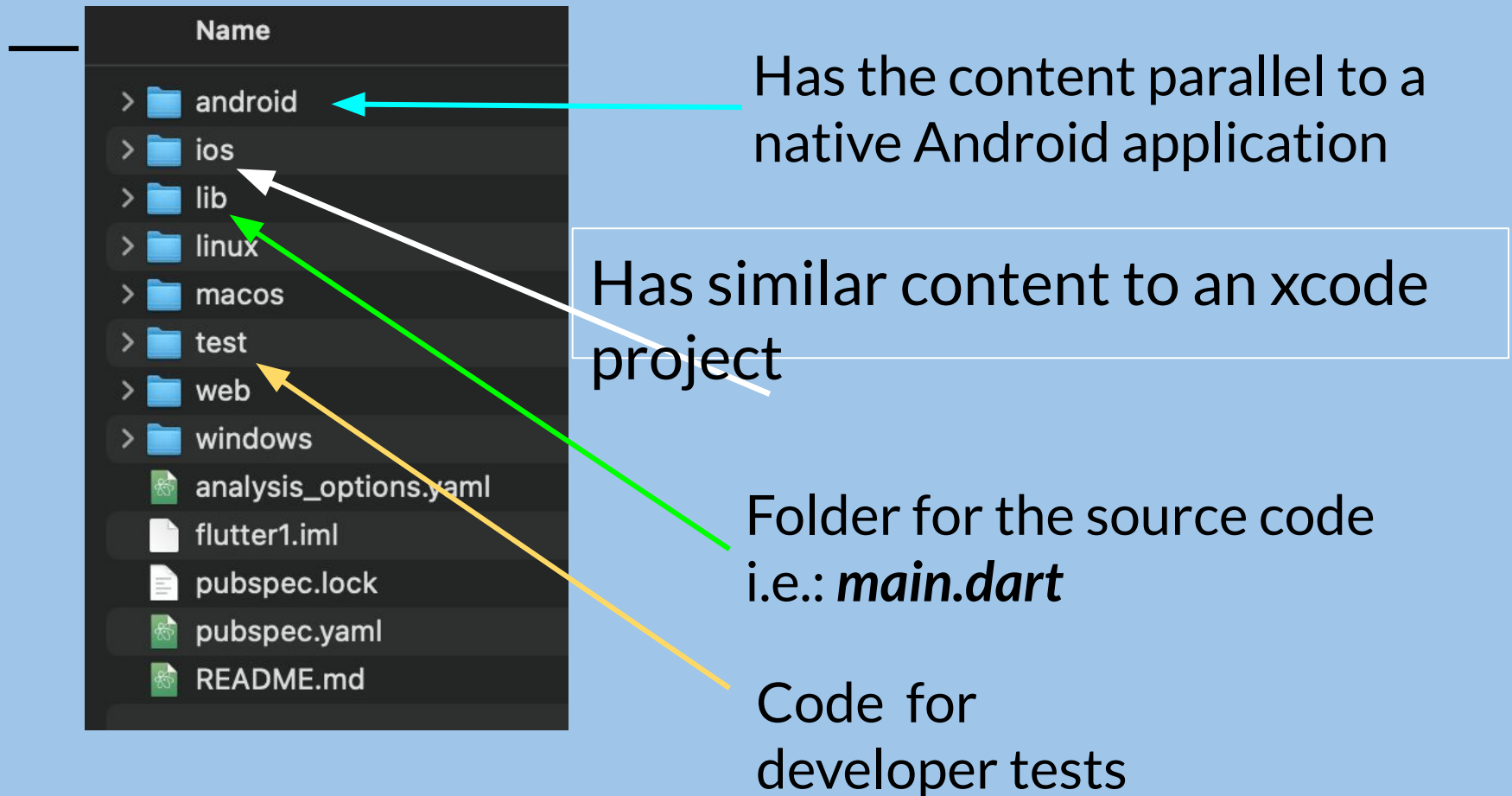
In order to run your application, type:

$ cd flutter1
$ flutter run

Your application code is in flutter1/lib/main.dart.

bash-5.2$
```

Creating from terminal...



More about project structure

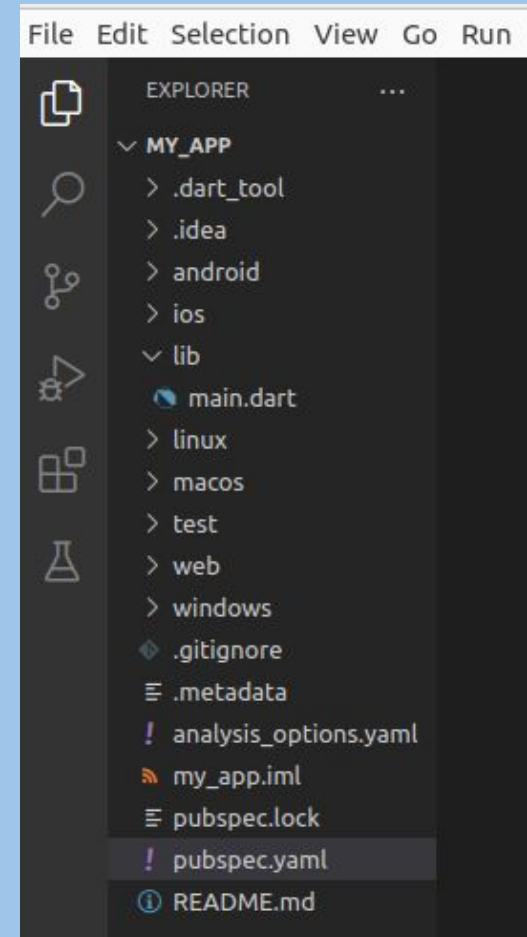
.gitignore for source management control

.metadata is a configuration file to leave to the tools

Analysis options are there to configure how the project is analysed by the static code analysis tools

Define rules/enable/disable rules - ie :

```
# prefer_single_quotes: true
```



More on analysis_options.yaml

analysis_options.yaml

How strict Flutter should be when analyzing code?

- At the beginning may be more relaxed
- At the publishing time may be more strict

i.e.: Set the rules to true at the end

<project_name.iml> - no interest for the developer

```
! analysis_options.yaml
1  include: package:flutter_lints/flutter.yaml
2
3  linter:
4    rules:
5      prefer_const_constructors: false
6      prefer_final_fields: false
7      use_key_in_widget_constructors: false
8      prefer_const_literals_to_create_immutables: false
9      prefer_const_constructors_in_immutables: false
10     avoid_print: false
```

Project structure... pubspec.yaml

The **pubspec.yaml** file specifies basic information about your app (among others):

- Version
- Dependencies
- Assets

Update the configuration as necessary
Similar to gradle file in native Android projects

```
! pubspec.yaml
1  name: my_app
2  description: A new Flutter proje
3
4  publish_to: 'none' # Remove this
5
6  version: 0.0.1+1
7
8  environment:
9    sdk: '>=2.19.4 <4.0.0'
10
11  dependencies:
12    flutter:
13      sdk: flutter
14
15    english_words: ^4.0.0
16    provider: ^6.0.0
17
18  dev_dependencies:
19    flutter_test:
20      sdk: flutter
21
22    flutter_lints: ^2.0.0
23
24  flutter:
25    uses-material-design: true
```

Project structure... pubspec.yaml

publish_to : **'none'** # prevent accidental publishing to public dart repositories

version : 0.0.1+1 # version number as major.minor.revision, +number is the build number

environment: sdk: '>=3.1.0 <4.0.0' # constraints on sdk versions

dependencies : # 'flutter pub upgrade --major-versions' - auto update; # 'flutter pub outdated' - check outdated

```
! pubspec.yaml
1  name: my_app
2  description: A new Flutter proje
3
4  publish_to: 'none' # Remove this
5
6  version: 0.0.1+1
7
8  environment:
9    sdk: '>=2.19.4 <4.0.0'
10
11  dependencies:
12    flutter:
13      sdk: flutter
14
15    english_words: ^4.0.0
16    provider: ^6.0.0
17
18  dev_dependencies:
19    flutter_test:
20      sdk: flutter
21
22    flutter_lints: ^2.0.0
23
24  flutter:
25    uses-material-design: true
```

Launch

—
Run the app with “flutter run”

OR

With the VSCode interface

<https://docs.flutter.dev/learn/pathway/tutorial/create-an-app>